

Code Size Optimization

Minimizing Executable Footprint

Measuring Code Size:

```
# Build and measure
arm-none-eabi-objcopy app.elf app.bin -O binary
ls -l app.bin # Shows actual flash usage

# Analyze size breakdown
arm-none-eabi-nm -S --size-sort app.elf | tail -20
```

ELF stands for **Executable and Linkable Format**. It is the standard file format for executables, object code, and libraries on Unix-like operating systems, including those running on microcontrollers. When you compile your C/C++ code for a microcontroller, the compiler initially produces an **ELF file** (e.g., app.elf). This file contains not only the raw machine code that will run on the chip but also extra metadata like debug symbols (function and variable names), section headers, and memory layout information.

Function-Level Analysis:

```
# Identify large functions
arm-none-eabi-objdump -S app.o | grep "function_name" -A 50
```

Optimization Strategies:

- Use -Os compiler flag for size optimization
- Remove debug symbols from production builds
- Enable link-time optimization (LTO)
- Manually optimize large functions identified by analysis

Framework Tuning:

```
// Reduce framework constants
#define TFLITE_REGISTRATIONS_MAX 16 // Default: 128
#define kStackDataAllocatorSize 2048 // Adjust for application
```